

🔗 See Topic 12 → Graphs Masterclass

🎯 **GOAL:** Master Backtracking intuition, state space decision trees, the Choose → Explore → Undo (State Restoration) paradigm, duplicate pruning, and solve all major FAANG questions!

📄 REUSABLE BACKTRACKING UTILITIES & TEMPLATE

```
// Generic Choose → Explore → Undo Skeleton
void backtrack(int start, int[] nums, List<Integer> path, List<List<Integer>> res) {
    res.add(new ArrayList<>(path)); // Add snapshot of valid state
    for (int i = start; i < nums.length; i++) {
        // 1. CHOOSE
        path.add(nums[i]);
        // 2. EXPLORE
        backtrack(i + 1, nums, path, res);
        // 3. UNDO (State Restoration)
        path.remove(path.size() - 1);
    }
}
```

💡 **AHA MOMENT #1: BACKTRACKING IS DEPTH-FIRST SEARCH ON A DECISION TREE WITH STATE RESTORATION**

Backtracking systematically explores all possible candidates in a State Space Tree. When a candidate path leads to a **\*\*Dead End\*\*** or **\*\*Invalid Constraint\*\***, the algorithm **\*\*undoes the last choice\*\*** (backtracks) to restore state and try the next branch!

1 CORE BACKTRACKING TERMINOLOGY

- **Decision Tree / State Space:** Hierarchy of all possible choices.
- **Candidate Path (`path`):** List of choices made so far.
- **Pruning / Branch Cutting:** Stopping exploration early when constraints are violated.
- **State Restoration:** `path.remove(path.size() - 1)` to reset for the next branch!

2 BRUTE FORCE VS BACKTRACKING

| Feature          | Pure Brute Force                   | Backtracking (With Pruning)                         |
|------------------|------------------------------------|-----------------------------------------------------|
| Exploration      | Generates ALL full sequences first | Explores incrementally step-by-step                 |
| Constraint Check | Checked only at leaf level         | Checked at <b>EVERY decision node</b>               |
| Efficiency       | $O(N^N)$ Full Space Explosion      | ✅ <b>Prunes thousands of invalid branches early</b> |

📊 VISUALIZING STATE SPACE DECISION TREE (SUBSETS OF [1, 2])

**DECISION TREE FOR SUBSETS OF [1, 2]:** `[] / \ [1] []` (Choice 1: Include '1' vs Exclude) `/ \ \ [1,2] [1] [2] []` (Choice 2: Include '2' vs Exclude) Result Subsets: `[]`, `[1]`, `[2]`, `[1, 2]` (Total  $2^N = 4$  subsets!)

🚀 **KEY TAKEAWAY:** Reach for Backtracking whenever a problem asks for "All Combinations", "All Permutations", "All Subsets", or "Constraint Satisfaction"!

### 💡 AHA MOMENT #2: THE GOLDEN 3-STEP BACKTRACKING PARADIGM

Every backtracking algorithm in computer science follows this exact 3-step loop:

1. **CHOOSE:** `path.add(candidate)`
2. **EXPLORE:** `backtrack(nextState, path)` (Recursive Call)
3. **UNDO:** `path.remove(path.size() - 1)` (State Restoration for next iteration!)

### 🔧 UNIVERSAL REUSABLE JAVA BACKTRACKING TEMPLATE

```
public List<List<Integer>> backtrackTemplate(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}

private void backtrack(int[] nums, int start, List<Integer> path, List<List<Integer>> result) {
    // 1. BASE CASE / SOLUTION FOUND
    result.add(new ArrayList<>(path)); // Make DEEP COPY of path!

    for (int i = start; i < nums.length; i++) {
        // 2. PRUNING / CONSTRAINT CHECK
        if (shouldPrune(nums, i)) continue;

        // 3. STEP 1: CHOOSE
        path.add(nums[i]);

        // 4. STEP 2: EXPLORE (Recursion)
        backtrack(nums, i + 1, path, result);

        // 5. STEP 3: UNDO (State Restoration)
        path.remove(path.size() - 1);
    }
}
```

### ⚠️ CRITICAL FAANG BUG: SHALLOW PATH REFERENCE INSTEAD OF DEEP COPY

- ❌ **WRONG:** `result.add(path);` → Stores a reference to the single mutating `path` object! Result will contain empty lists at the end!
- ✅ **CORRECT:** `result.add(new ArrayList<>(path));` → Creates an independent snapshot of current path!

🔑 **KEY TAKEAWAY:** ALWAYS pass `new ArrayList<>(path)` when appending solutions to `result`!

- 💡 AHA MOMENT #3: PASSING `start` INDEX VS `visited[]` BOOLEAN ARRAY
- Subsets & Combinations (Order DOES NOT matter): `[1, 2]` is identical to `[2, 1]`. Pass `start` index to recursion (`i + 1`) to only pick elements moving forward!
  - Permutations (Order DOES matter): `[1, 2]` is DIFFERENT from `[2, 1]`. Start loop at index `0` every time and use a `boolean[] visited` array!

 SUBSETS VS COMBINATIONS VS PERMUTATIONS MATRIX

| Pattern Type | Order Matters? | Loop Start Index             | Duplicate Skipping Rule                                                                             | Output Count   |
|--------------|----------------|------------------------------|-----------------------------------------------------------------------------------------------------|----------------|
| Subsets      | ❌ No           | <code>i + 1</code> (Forward) | <code>if (i &gt; start &amp;&amp; nums[i] == nums[i-1]) continue;</code>                            | $2^N$          |
| Combinations | ❌ No           | <code>i + 1</code> (Forward) | <code>if (i &gt; start &amp;&amp; nums[i] == nums[i-1]) continue;</code>                            | $\binom{N}{K}$ |
| Permutations | ✅ YES          | <code>0</code> (Every time)  | <code>if (visited[i]    (i &gt; 0 &amp;&amp; nums[i] == nums[i-1] &amp;&amp; !visited[i-1]))</code> | $N!$           |

🚩 DUPLICATE SKIPPING LAW: Sort input array first (`Arrays.sort(nums)`), then check `if (i > start && nums[i] == nums[i-1]) continue;!`

**💡 AHA MOMENT #4: IN-PLACE MUTATION & UN-MUTATION FOR GRIDS**

In Grid Backtracking (Word Search), avoid allocating extra `visited[][]` matrices!

1. **Choose:** Save `char temp = board[r][c]; board[r][c] = '#';`
2. **Explore:** Recurse to 4 neighbor cells.
3. **Undo:** Restore `board[r][c] = temp;`!

**1 N-QUEENS  $O(1)$  CONSTRAINT CHECKING**

```
// Track attacked columns and diagonals
boolean[] cols = new boolean[n];
boolean[] diag1 = new boolean[2 * n]; // r + c
boolean[] diag2 = new boolean[2 * n]; // r - c + n

boolean isSafe(int r, int c, int n) {
    return !cols[c] && !diag1[r + c] && !diag2[r - c + n];
}
```

**2 GRID IN-PLACE BACKTRACKING**

```
char temp = board[r][c];
board[r][c] = '#'; // Mark visited (Choose)

// Explore 4 directions
boolean found = dfs(r+1, c) || dfs(r-1, c) ||
                dfs(r, c+1) || dfs(r, c-1);

board[r][c] = temp; // Un-mark visited (Undo!)
```

🚀 **KEY TAKEAWAY:** In-place grid cell mutation saves  $O(M \cdot N)$  memory and ensures high-performance grid DFS!

💡 **AHA MOMENT #5: PRUNING SAVES MILLIONS OF RECURSIVE CALLS**

Without pruning, N-Queens has a search space of  $N^N$  ( $8^8 = 16,777,216$  states!).

By checking constraints AT EVERY CHOICE and skipping invalid branches early, pruning cuts exploration down to ONLY 2,057 states ( $10,000\times$  faster)!

**1 BOUNDARY & TARGET OVERFLOW PRUNING**

- **Target Overflow:** In Combination Sum, if `target < 0`, stop searching immediately (`return;`).
- **Early Exit:** Stop recursion as soon as 1 valid answer is found (`return 'boolean'`).

**2 DUPLICATE BRANCH PRUNING**

- Sort input array first (`Arrays.sort(nums)`).
- Skip identical elements at the same recursion depth level: `if (i > start && nums[i] == nums[i-1]) continue;`

**3 CODE MECHANICS: DUPLICATE PRUNING LOGIC**

```
// Sort input to bring identical elements together
Arrays.sort(candidates);

for (int i = start; i < candidates.length; i++) {
    // PRUNE: If current candidate is identical to previous sibling at SAME depth level, skip!
    if (i > start && candidates[i] == candidates[i - 1]) continue;

    if (candidates[i] > target) break; // PRUNE: Array is sorted, so subsequent items also exceed target!

    path.add(candidates[i]);
    backtrack(candidates, target - candidates[i], i + 1, path, res);
    path.remove(path.size() - 1);
}
```

✂ **PRUNING RULE:** Always place `if (remainingTarget < 0) return;` at the VERY TOP of your recursive function!

 BACKTRACKING INTERVIEW DECISION TREE (MERMAID)

 PRINTABLE BACKTRACKING CHEAT SHEET

| Pattern Type      | Target Problem          | Loop Index & Recursion                                                  | Time Complexity          |
|-------------------|-------------------------|-------------------------------------------------------------------------|--------------------------|
| Subsets           | Subsets I (LC 78)       | <code>backtrack(i + 1, path)</code>                                     | $O(N \cdot 2^N)$         |
| Subsets with Dups | Subsets II (LC 90)      | Sort + <code>i &gt; start &amp; nums[i] == nums[i-1]</code>             | $O(N \cdot 2^N)$         |
| Combinations      | Combination Sum (LC 39) | Pass <code>i</code> for reuse, <code>i + 1</code> for single use        | $O(N^{\frac{T}{M}})$     |
| Permutations      | Permutations (LC 46)    | Loop <code>i = 0</code> + <code>visited[i]</code> boolean array         | $O(N \cdot N!)$          |
| Grid Search       | Word Search (LC 79)     | In-place <code>board[r][c] = '#'</code> and restore                     | $O(M \cdot N \cdot 4^L)$ |
| Constraint Board  | N-Queens (LC 51)        | Track <code>cols[]</code> , <code>diag1[]</code> , <code>diag2[]</code> | $O(N!)$                  |

 **FAANG COACH TIP:** Print this page and review it 10 minutes before your technical interview!



## 1. SUBSETS TEMPLATE (LC 78 / 90)

```
void backtrackSubsets(int[] nums, int start, List<Integer> path,
List<List<Integer>> res) {
    res.add(new ArrayList<>(path));
    for (int i = start; i < nums.length; i++) {
        if (i > start && nums[i] == nums[i - 1]) continue; // Skip
dups
        path.add(nums[i]);
        backtrackSubsets(nums, i + 1, path, res);
        path.remove(path.size() - 1);
    }
}
```

## 2. COMBINATIONS TEMPLATE (LC 39 / 40)

```
void backtrackCombo(int[] nums, int target, int start, List<Integer>
path, List<List<Integer>> res) {
    if (target == 0) { res.add(new ArrayList<>(path)); return; }
    for (int i = start; i < nums.length; i++) {
        if (nums[i] > target) break; // Prune target overflow
        if (i > start && nums[i] == nums[i - 1]) continue; // Skip
dups
        path.add(nums[i]);
        backtrackCombo(nums, target - nums[i], i + 1, path, res); //
i+1 for single use
        path.remove(path.size() - 1);
    }
}
```

🚀 **REUSABLE CODE:** Copy-paste these templates directly into your interview whiteboard or IDE!

**3. PERMUTATIONS TEMPLATE (LC 46 / 47)**

```
void backtrackPerm(int[] nums, boolean[] vis, List<Integer> path,
List<List<Integer>> res) {
    if (path.size() == nums.length) { res.add(new ArrayList<>(path));
return; }
    for (int i = 0; i < nums.length; i++) {
        if (vis[i]) continue;
        if (i > 0 && nums[i] == nums[i-1] && !vis[i-1]) continue; //
Skip dups
        vis[i] = true; path.add(nums[i]);
        backtrackPerm(nums, vis, path, res);
        path.remove(path.size() - 1); vis[i] = false;
    }
}
```

**4. GRID SEARCH TEMPLATE (LC 79)**

```
boolean dfsGrid(char[][] b, String w, int r, int c, int idx) {
    if (idx == w.length()) return true;
    if (r < 0 || r ≥ b.length || c < 0 || c ≥ b[0].length || b[r][c]
≠ w.charAt(idx)) return false;
    char tmp = b[r][c]; b[r][c] = '#'; // Choose
    boolean res = dfsGrid(b, w, r+1, c, idx+1) || dfsGrid(b, w, r-1,
c, idx+1) ||
                    dfsGrid(b, w, r, c+1, idx+1) || dfsGrid(b, w, r, c-
1, idx+1);
    b[r][c] = tmp; return res; // Undo!
}
```

🚀 **KEY TAKEAWAY:** Memorize these core templates and you can solve 95% of FAANG Backtracking problems!



## 1. LEETCODE 17 — LETTER COMBINATIONS OF A PHONE NUMBER

Medium ★★★★★ Digit Mapping Recursion Amazon Meta

**Problem:**

Return all possible letter combinations that input digits could represent on a phone keypad.

```
class Solution {
    private static final String[] MAP = {
        "", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"
    };
    public List<String> letterCombinations(String digits) {
        List<String> res = new ArrayList<>();
        if (digits.isEmpty()) return res;
        backtrack(digits, 0, new StringBuilder(), res);
        return res;
    }
    private void backtrack(String digits, int idx, StringBuilder path,
        List<String> res) {
        if (idx == digits.length()) { res.add(path.toString()); return; }
        String letters = MAP[digits.charAt(idx) - '0'];
        for (char c : letters.toCharArray()) {
            path.append(c); // Choose
            backtrack(digits, idx + 1, path, res); // Explore
            path.deleteCharAt(path.length() - 1); // Undo!
        }
    }
}
```

💡 **AHA MOMENT (LC 17):** Using `StringBuilder` with `append()` and `deleteCharAt()` manages character string state without creating new string objects!



## 2. LEETCODE 78 — SUBSETS (POWER SET)

Medium ★★★★★ Power Set Generation Amazon Meta

**Problem:**

Return all possible subsets (the power set) of unique elements.

```
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), res);
    return res;
}

private void backtrack(int[] nums, int start, List<Integer> path,
    List<List<Integer>> res) {
    res.add(new ArrayList<>(path));
    for (int i = start; i < nums.length; i++) {
        path.add(nums[i]);
        backtrack(nums, i + 1, path, res);
        path.remove(path.size() - 1);
    }
}
```

## 3. LEETCODE 90 — SUBSETS II (WITH DUPLICATES)

Medium ★★★★★ Duplicate Branch Pruning Amazon

**Problem:**

Return all possible subsets of array containing duplicate numbers.

```
public List<List<Integer>> subsetsWithDup(int[] nums) {
    Arrays.sort(nums); // Sort first for duplicate check!
    List<List<Integer>> res = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), res);
    return res;
}

private void backtrack(int[] nums, int start, List<Integer> path,
    List<List<Integer>> res) {
    res.add(new ArrayList<>(path));
    for (int i = start; i < nums.length; i++) {
        if (i > start && nums[i] == nums[i - 1]) continue; // Skip dups!
        path.add(nums[i]);
        backtrack(nums, i + 1, path, res);
        path.remove(path.size() - 1);
    }
}
```

💡 **AHA MOMENT (LC 90):** Checking `i > start && nums[i] == nums[i-1]` skips duplicate choices at the SAME decision level while allowing them across different depth levels!



## 4. LEETCODE 39 — COMBINATION SUM I

Medium ★★★★★ Unlimited Re-Use Target Sum Amazon Meta

**Problem:**

Return combinations summing to `target` (same element can be chosen unlimited times).

```
public List<List<Integer>> combinationSum(int[] candidates, int target)
{
    List<List<Integer>> res = new ArrayList<>();
    Arrays.sort(candidates);
    backtrack(candidates, target, 0, new ArrayList<>(), res);
    return res;
}

private void backtrack(int[] c, int target, int start, List<Integer>
path, List<List<Integer>> res) {
    if (target == 0) { res.add(new ArrayList<>(path)); return; }
    for (int i = start; i < c.length; i++) {
        if (c[i] > target) break; // Early target overflow pruning
        path.add(c[i]);
        backtrack(c, target - c[i], i, path, res); // Pass 'i' for
reuse!
        path.remove(path.size() - 1);
    }
}
```

## 5. LEETCODE 40 — COMBINATION SUM II

Medium ★★★★★ Single-Use + Duplicate Pruning Amazon

**Problem:**

Return combinations summing to `target` (each element used AT MOST ONCE).

```
public List<List<Integer>> combinationSum2(int[] candidates, int target)
{
    List<List<Integer>> res = new ArrayList<>();
    Arrays.sort(candidates);
    backtrack(candidates, target, 0, new ArrayList<>(), res);
    return res;
}

private void backtrack(int[] c, int target, int start, List<Integer>
path, List<List<Integer>> res) {
    if (target == 0) { res.add(new ArrayList<>(path)); return; }
    for (int i = start; i < c.length; i++) {
        if (c[i] > target) break;
        if (i > start && c[i] == c[i - 1]) continue; // Skip dups
        path.add(c[i]);
        backtrack(c, target - c[i], i + 1, path, res); // Pass 'i+1' for
single-use!
        path.remove(path.size() - 1);
    }
}
```

💡 **AHA MOMENT (LC 39 vs LC 40):** Pass `i` to recursion for unlimited element reuse; pass `i + 1` for single element use!

## 6. LEETCODE 46 — PERMUTATIONS

 **Medium ★★★★★** Order-Sensitive Visited Tracking [Amazon](#) [Meta](#)

### Problem:

Return all possible permutations of an array of distinct integers.

```
public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    backtrack(nums, new boolean[nums.length], new ArrayList<>(), res);
    return res;
}

private void backtrack(int[] nums, boolean[] vis, List<Integer> path,
List<List<Integer>> res) {
    if (path.size() == nums.length) { res.add(new ArrayList<>(path));
    return; }
    for (int i = 0; i < nums.length; i++) {
        if (vis[i]) continue;
        vis[i] = true; path.add(nums[i]);
        backtrack(nums, vis, path, res);
        path.remove(path.size() - 1); vis[i] = false;
    }
}
```

## 7. LEETCODE 47 — PERMUTATIONS II (WITH DUPLICATES)

 **Medium ★★★★★** Permutation Duplicate Pruning [Amazon](#)

### Problem:

Return unique permutations of array containing duplicate numbers.

```
public List<List<Integer>> permuteUnique(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> res = new ArrayList<>();
    backtrack(nums, new boolean[nums.length], new ArrayList<>(), res);
    return res;
}

private void backtrack(int[] nums, boolean[] vis, List<Integer> path,
List<List<Integer>> res) {
    if (path.size() == nums.length) { res.add(new ArrayList<>(path));
    return; }
    for (int i = 0; i < nums.length; i++) {
        if (vis[i]) continue;
        if (i > 0 && nums[i] == nums[i - 1] && !vis[i - 1]) continue; //
Skip duplicate branch
        vis[i] = true; path.add(nums[i]);
        backtrack(nums, vis, path, res);
        path.remove(path.size() - 1); vis[i] = false;
    }
}
```

 **MEMORY BOX:** In Permutations II, `!vis[i-1]` ensures that duplicate elements are inserted **ONLY** in strict left-to-right order!



## 8. LEETCODE 131 — PALINDROME PARTITIONING

Medium ★★★★★ Substring Palindrome Check [Amazon](#) [Meta](#)**Problem:**

Partition string 's' such that every substring of partition is a palindrome.

```
public List<List<String>> partition(String s) {
    List<List<String>> res = new ArrayList<>();
    backtrack(s, 0, new ArrayList<>(), res);
    return res;
}

private void backtrack(String s, int start, List<String> path,
    List<List<String>> res) {
    if (start == s.length()) { res.add(new ArrayList<>(path)); return; }
    for (int end = start; end < s.length(); end++) {
        if (isPalindrome(s, start, end)) {
            path.add(s.substring(start, end + 1));
            backtrack(s, end + 1, path, res);
            path.remove(path.size() - 1);
        }
    }
}

private boolean isPalindrome(String s, int l, int r) {
    while (l < r) if (s.charAt(l++) != s.charAt(r--)) return false;
    return true;
}
```

## 9. LEETCODE 79 — WORD SEARCH

Medium ★★★★★ Grid DFS In-Place Cell Mutation [Amazon](#) [Meta](#)**Problem:**

Determine if target word exists in 2D grid matrix of letters.

```
public boolean exist(char[][] board, String word) {
    for (int r = 0; r < board.length; r++) {
        for (int c = 0; c < board[0].length; c++) {
            if (dfs(board, word, r, c, 0)) return true;
        }
    }
    return false;
}

private boolean dfs(char[][] b, String w, int r, int c, int idx) {
    if (idx == w.length()) return true;
    if (r < 0 || r ≥ b.length || c < 0 || c ≥ b[0].length || b[r][c]
        != w.charAt(idx)) return false;
    char tmp = b[r][c]; b[r][c] = '#'; // Mark visited
    boolean found = dfs(b, w, r+1, c, idx+1) || dfs(b, w, r-1, c, idx+1)
        || dfs(b, w, r, c+1, idx+1) || dfs(b, w, r, c-1,
        idx+1);
    b[r][c] = tmp; // Restore cell state!
    return found;
}
```

**AHA MOMENT (LC 79):** Restoring 'b[r][c] = tmp' right after DFS exploration enables trying other valid grid paths!

## 10. LEETCODE 51 — N-QUEENS

 **Hard ★★★★★** Board Placement Constraint Check [Amazon](#) [Meta](#)

### Problem:

Place  $N$  queens on an  $N \times N$  chessboard such that no two queens attack each other.

```
class Solution {
    public List<List<String>> solveNQueens(int n) {
        List<List<String>> res = new ArrayList<>();
        char[][] board = new char[n][n];
        for (char[] row : board) Arrays.fill(row, '.');
        backtrack(board, 0, new boolean[n], new boolean[2*n], new
        boolean[2*n], res);
        return res;
    }
    private void backtrack(char[][] b, int r, boolean[] cols, boolean[]
    d1, boolean[] d2, List<List<String>> res) {
        int n = b.length;
        if (r == n) {
            List<String> path = new ArrayList<>();
            for (char[] row : b) path.add(new String(row));
            res.add(path); return;
        }
        for (int c = 0; c < n; c++) {
            if (cols[c] || d1[r + c] || d2[r - c + n]) continue;
            b[r][c] = 'Q'; cols[c] = d1[r + c] = d2[r - c + n] = true;
            backtrack(b, r + 1, cols, d1, d2, res);
            b[r][c] = '.'; cols[c] = d1[r + c] = d2[r - c + n] = false;
        }
    }
}
```

## 11. LEETCODE 37 — SUDOKU SOLVER

 **Hard ★★★★★** 9×9 Grid Cell Choice [Google](#)

### Problem:

Solve a Sudoku puzzle by filling empty cells '.' with digits '1'-'9'.

```
public void solveSudoku(char[][] board) { solve(board); }
private boolean solve(char[][] b) {
    for (int r = 0; r < 9; r++) {
        for (int c = 0; c < 9; c++) {
            if (b[r][c] == '.') {
                for (char ch = '1'; ch <= '9'; ch++) {
                    if (isValid(b, r, c, ch)) {
                        b[r][c] = ch;
                        if (solve(b)) return true;
                        b[r][c] = '.'; // Backtrack
                    }
                }
                return false;
            }
        }
    }
    return true;
}
private boolean isValid(char[][] b, int r, int c, char ch) {
    for (int i = 0; i < 9; i++) {
        if (b[r][i] == ch || b[i][c] == ch) return false;
        if (b[3 * (r / 3) + i / 3][3 * (c / 3) + i % 3] == ch) return
        false;
    }
    return true;
}
```

 **FAANG COACH TIP:** Fast  $O(1)$  diagonal lookups ' $d1[r + c]$ ' and ' $d2[r - c + n]$ ' make N-Queens lightning fast!

## 12. LEETCODE 212 — WORD SEARCH II

**Hard ★★★★★** [Trie + Grid Backtracking Pruning](#) [Amazon](#) [Meta](#) [Google](#)

### Problem:

Find all dictionary words present on a 2D board.

```
class Solution {
    class TrieNode { TrieNode[] children = new TrieNode[26]; String
word; }
    public List<String> findWords(char[][] board, String[] words) {
        TrieNode root = new TrieNode();
        for (String w : words) {
            TrieNode curr = root;
            for (char c : w.toCharArray()) {
                int idx = c - 'a';
                if (curr.children[idx] == null) curr.children[idx] = new
TrieNode();
                curr = curr.children[idx];
            }
            curr.word = w;
        }
        List<String> res = new ArrayList<>();
        for (int r = 0; r < board.length; r++) {
            for (int c = 0; c < board[0].length; c++) dfs(board, r, c,
root, res);
        }
        return res;
    }
    private void dfs(char[][] b, int r, int c, TrieNode node,
List<String> res) {
        char ch = b[r][c];
        if (ch == '#' || node.children[ch - 'a'] == null) return;
        node = node.children[ch - 'a'];
        if (node.word != null) { res.add(node.word); node.word = null; }
        b[r][c] = '#';
        int[][] dirs = {{0,1},{1,0},{0,-1},{-1,0}};
        for (int[] d : dirs) {
            int nr = r + d[0], nc = c + d[1];
            if (nr >= 0 && nr < b.length && nc >= 0 && nc < b[0].length)
dfs(b, nr, nc, node, res);
        }
        b[r][c] = ch; // Backtrack
    }
}
```

## 13. LEETCODE 980 — UNIQUE PATHS III

**Hard ★★★★★** [Hamiltonian Grid Path Backtrack](#) [Google](#)

### Problem:

Paths from start cell to end cell walking over EVERY non-obstacle cell EXACTLY ONCE.

```
public int uniquePathsIII(int[][] grid) {
    int startR = 0, startC = 0, empty = 1;
    for (int r = 0; r < grid.length; r++) {
        for (int c = 0; c < grid[0].length; c++) {
            if (grid[r][c] == 1) { startR = r; startC = c; }
            else if (grid[r][c] == 0) empty++;
        }
    }
    return dfs(grid, startR, startC, empty);
}
private int dfs(int[][] g, int r, int c, int count) {
    if (r < 0 || r >= g.length || c < 0 || c >= g[0].length || g[r][c]
== -1) return 0;
    if (g[r][c] == 2) return count == 0 ? 1 : 0;
    g[r][c] = -1; // Choose
    int paths = dfs(g, r+1, c, count-1) + dfs(g, r-1, c, count-1) +
dfs(g, r, c+1, count-1) + dfs(g, r, c-1, count-1);
    g[r][c] = 0; // Undo!
    return paths;
}
```

**💡 AHA MOMENT (LC 980):** Countdown variable `empty` ensures every valid path visits all non-obstacle grid cells before reaching target `2`!

### ★ THE 4 GOLDEN LAWS OF BACKTRACKING

1. **State Restoration Law:** Always undo choices after recursive calls: `path.remove(path.size() - 1)`.
2. **Deep Copy Result Law:** Always store deep copies `result.add(new ArrayList<>(path))`.
3. **Duplicate Skipping Rule:** Sort input array first, then check `if (i > start && nums[i] == nums[i-1]) continue;`.
4. **Subsets vs Permutations:** Pass `i + 1` for Subsets/Combinations. Use `boolean[] visited` for Permutations!

### 🚫 WHEN BACKTRACKING FAILS (DO NOT USE!)

- **Overlapping Subproblems without Path Tracking:** Searching min cost without unique paths → Use **\*\*Dynamic Programming\*\***!
- **Local Choice Always Optimal:** If greedy choice gives global optimum → Use **\*\*Greedy\*\***!

### 📁 NEETCODE & BLIND 75 CONFIDENCE TRACKER

|                                       |                                       |                                     |
|---------------------------------------|---------------------------------------|-------------------------------------|
| 🔹 <b>Med</b> LC 17 Phone Numbers      | 🔹 <b>Med</b> LC 78 Subsets I          | 🔹 <b>Med</b> LC 90 Subsets II       |
| 🔹 <b>Med</b> LC 39 Combination Sum I  | 🔹 <b>Med</b> LC 40 Combination Sum II | 🔹 <b>Med</b> LC 46 Permutations I   |
| 🔹 <b>Med</b> LC 47 Permutations II    | 🔹 <b>Med</b> LC 131 Palindrome Part   | 🔹 <b>Med</b> LC 79 Word Search      |
| 🔴 <b>Hard</b> LC 51 N-Queens          | 🔴 <b>Hard</b> LC 37 Sudoku Solver     | 🔴 <b>Hard</b> LC 212 Word Search II |
| 🔴 <b>Hard</b> LC 980 Unique Paths III |                                       |                                     |

👉 **Next Master Topic: Topic 14 → Dynamic Programming Masterclass**

🎉 **TOPIC 13 BACKTRACKING MASTERCLASS COMPLETE!** Turn the page to **Topic 14: Dynamic Programming Masterclass!**